1. What Is Machine Learning?

Machine learning is the study of algorithms that improve their performance on a task by learning from data rather than by following explicitly programmed rules. A program is said to learn from experience if its performance on a task, measured by some metric, improves as it sees more examples.

There are three broad families of machine learning. In supervised learning, the training data includes the correct answer, called a label, for every example, and the model learns to predict labels for new inputs. In unsupervised learning, the data has no labels and the model looks for structure on its own, for example by grouping similar items together in a process called clustering. In reinforcement learning, an agent interacts with an environment, receives rewards or penalties for its actions, and learns a policy that maximizes its cumulative reward over time.

Every machine learning project follows a similar workflow: collect data, choose features, select a model, train it, evaluate it on data it has not seen, and finally deploy and monitor it.

2. Supervised Learning: Classification and Regression

Supervised learning problems fall into two main types. Classification predicts a discrete category, such as whether an email is spam or not spam. Regression predicts a continuous numeric value, such as the price of a house given its size and location.

Each example is described by features, which are the measurable input variables, and a label, which is the target the model must predict. Before training, the dataset is split into a training set, which the model learns from, and a test set, which is held back and used only once at the end to estimate how well the model generalizes to new data. A common split is 80 percent for training and 20 percent for testing.

Using the test set repeatedly to make design decisions leaks information about it into the model. To tune settings called hyperparameters without touching the test set, practitioners hold out a third portion of the data called the validation set.

3. Loss Functions and Gradient Descent

A loss function measures how wrong a model's predictions are. For regression, a common choice is the mean squared error, which is the average of the squared differences between predictions and true values. Training a model means finding the parameters that minimize the loss.

Gradient descent is an iterative optimization algorithm for minimizing a loss function. At each step it computes the gradient of the loss with respect to the parameters, which points in the direction of steepest increase, and then moves the parameters a small step in the opposite direction. The size of that step is controlled by the learning rate.

Choosing the learning rate matters. If the learning rate is too large, the updates overshoot the minimum and the loss can oscillate or even diverge. If it is too small, training converges very slowly. Stochastic gradient descent estimates the gradient from a single example or a small mini-batch instead of the whole dataset, which makes each update much cheaper.

4. Overfitting, Underfitting and Regularization

Overfitting happens when a model learns the noise and quirks of the training data so closely that it performs well on the training set but poorly on new data. The telltale sign is a large gap between low training error and high test error. Underfitting is the opposite problem: the model is too simple to capture the real pattern, so it performs poorly on both the training set and the test set.

This tension is described by the bias-variance tradeoff. High bias means the model makes strong simplifying assumptions and underfits. High variance means the model is very sensitive to the particular training sample and overfits. Increasing model complexity usually lowers bias but raises variance.

Regularization reduces overfitting by adding a penalty on large parameter values to the loss function. L2 regularization, also called ridge regression, adds the sum of the squared weights and shrinks all weights toward zero. L1 regularization, also called lasso, adds the sum of the absolute values of the weights and can drive some weights exactly to zero, which performs a form of feature selection. Getting more training data and stopping training early are other ways to reduce overfitting.

5. Evaluating Models

Cross-validation gives a more reliable estimate of performance than a single train-test split. In k-fold cross-validation, the data is divided into k equal folds. The model is trained k times, each time holding out a different fold for validation and training on the remaining k minus one folds, and the k scores are averaged.

For classification, accuracy is the fraction of predictions that are correct, but it can be misleading when classes are imbalanced. A confusion matrix counts true positives, false positives, true negatives and false negatives. Precision is the number of true positives divided by the number of predicted positives, so it answers the question of how many flagged items were truly positive. Recall is the number of true positives divided by the number of actual positives, so it answers how many of the real positives were found. The F1 score is the harmonic mean of precision and recall.

For regression, common metrics are mean absolute error and root mean squared error, which is the square root of the mean squared error and is expressed in the same units as the target.

6. Neural Networks and Backpropagation

An artificial neural network is built from layers of simple units called neurons. Each neuron computes a weighted sum of its inputs, adds a bias, and passes the result through a nonlinear activation function such as ReLU or the sigmoid. Without nonlinear activations, stacking layers would still produce only a linear model.

Backpropagation is the algorithm used to train neural networks. A forward pass computes the predictions and the loss. A backward pass then applies the chain rule of calculus to compute the gradient of the loss with respect to every weight, layer by layer, starting from the output. An optimizer such as gradient descent uses those gradients to update the weights.

Deep networks have many hidden layers and can learn hierarchical features, but they need large amounts of data and computing power and are prone to overfitting without regularization techniques such as dropout, which randomly disables a fraction of neurons during each training step.